

IBM Translation Models in LAPS

Overview

This document explains how the original LAPS paper (Wong et al., ICML 2021) used IBM Model 4—a classical statistical machine translation model from the 1990s—to align natural language descriptions with programs.

Part 1: What Are IBM Translation Models?

The Noisy Channel Framework

IBM Models treat translation as **recovering a signal from noise**:

$P(\text{description} \mid \text{program}) = \text{Translation Model}$

"Given this program, what's the probability of this description?"

The Model Family

Model	What It Learns	Key Innovation
IBM Model 1	Word-to-word translation probabilities	Ignores word order; just counts co-occurrences
IBM Model 2	+ Position-dependent alignment	"Word at position 3 aligns to position 5"
IBM Model 3	+ Fertility (one word $\rightarrow$ many words)	"map" might generate both "apply" and "function"
IBM Model 4	+ Sophisticated distortion	Related words move together

IBM Model 1: The Core Idea

Goal: Learn $t(e|f)$ = probability that source word f translates to target word e

Training: Given parallel sentences (not word-aligned), use EM:

E-step: Estimate which words align based on current $t(e|f)$

M-step: Update $t(e|f)$ based on estimated alignments

Example:

Program tokens: [map, lambda, add, 1]

Description: "add one to each element"

After training:

$t(\text{"add"} \mid \text{add}) = 0.8$

$t(\text{"one"} \mid 1) = 0.7$

$t(\text{"each"} \mid \text{map}) = 0.5$

$t(\text{"element"} \mid \text{lambda}) = 0.3$

Mathematical Formulation

For source sentence f (program) and target sentence e (description):

$$\begin{aligned} P(e \mid f) &= \sum_a P(e, a \mid f) \\ &= \sum_a t(e \mid f_{\{a\}}) \times \text{alignment_prob}(a) \end{aligned}$$

Where: - **a** is an alignment (which source word each target word came from) - **t(e|f)** is the translation probability - IBM Model 1 assumes uniform alignment_prob - IBM Model 4 models alignment_prob with position-dependent distortion

Part 2: How LAPS Uses IBM Model 4

The Key Insight

LAPS treats **programs as one language** and **descriptions as another language**, then uses IBM Model 4 to learn the translation probabilities between them.

What Gets Aligned

```
Program (linearized):    [filter, lambda, eq, color, red]
Description:             "keep only the red items"
```

IBM Model 4 learns:

```
P("keep" | filter) = 0.6
P("only" | filter) = 0.4
P("red" | red) = 0.9
P("items" | lambda) = 0.3
```

The Translation Model $T(d | p, L)$

```
T(description | program, library) =
    t(description | program_token_{a}) * distortion(a)
```

Integration into the Joint Objective

LAPS extends DreamCoder's objective:

```
P(program | examples, description, grammar)
    P(examples | program)      ← Likelihood
    × T(description | program) ← Translation model
    × P(program | grammar)     ← Grammar prior
```

How It Guides Search

```
def reweight_with_translation(primitive_probs, description, translation_model):
    for primitive in primitives:
        translation_boost = sum(
            T(word | primitive)
            for word in description.split()
        )
        primitive_probs[primitive] *= translation_boost
    return normalize(primitive_probs)
```

Part 3: Where Do Descriptions Come From?

In LAPS: Human-Provided

LAPS used **human annotations** for training tasks: - String editing: “remove the last word”, “capitalize first letter” - Image composition: “red circle above blue square” - Scene reasoning: “there is something small”

The Bootstrap Problem

LAPS doesn't generate descriptions—it consumes them:

Training: examples + description $\rightarrow$ program

Testing: examples only (or with description if available)

Part 4: IBM Model 4 vs LLM Approach (LILO)

Aspect	IBM Model 4 (LAPS)	LLM (LILO AutoDoc)
Direction	description $\rightarrow$ program	program $\rightarrow$ description
Training data	Parallel pairs	Zero-shot (pretrained)
Interpretability	High (word alignments)	Low (black box)
Compute cost	Very low	High (API calls)
Novel descriptions	Poor	Good
Novel programs	N/A	Good
Consistency	Deterministic	Non-deterministic

References

- Brown et al. (1993). The Mathematics of Statistical Machine Translation. Computational Linguistics.
 - Wong et al. (2021). Leveraging Language to Learn Program Abstractions and Search Heuristics. ICML.
 - Grand et al. (2024). LILO: Learning Interpretable Libraries by Compressing and Documenting Code. ICLR.
-

Generated as part of /research-ultrathink analysis December 2024